

13. GENERAL ALGORITHMS

Some algorithms **out of the context** of ADTs

Graphs:

- Depth-first search and breadth-first search

- Shortest path

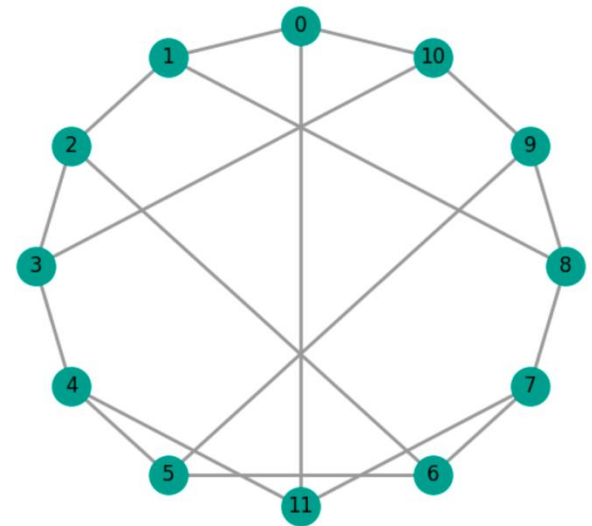
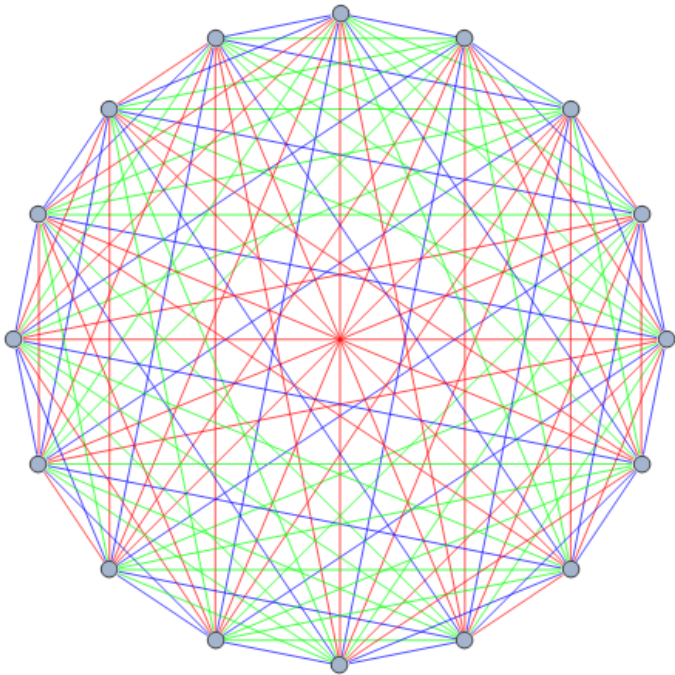
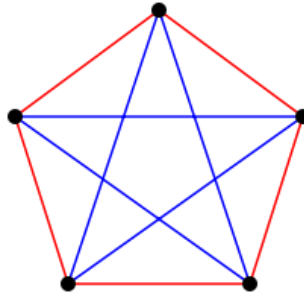
- Bin packing

Introduction –why these slides

- There are some common algorithms that you might be asked about in a job interview (for example)
- Many of these are graph-based
- Just a few slides so far, may extend in future years
- I will not go through the slides completely. But the **bin packing slides are examinable** (rest are not).

Graphs

- A graph G consists of a set of vertices V and a set of edges E
- A tree is a graph that has no cycles
- Examples:



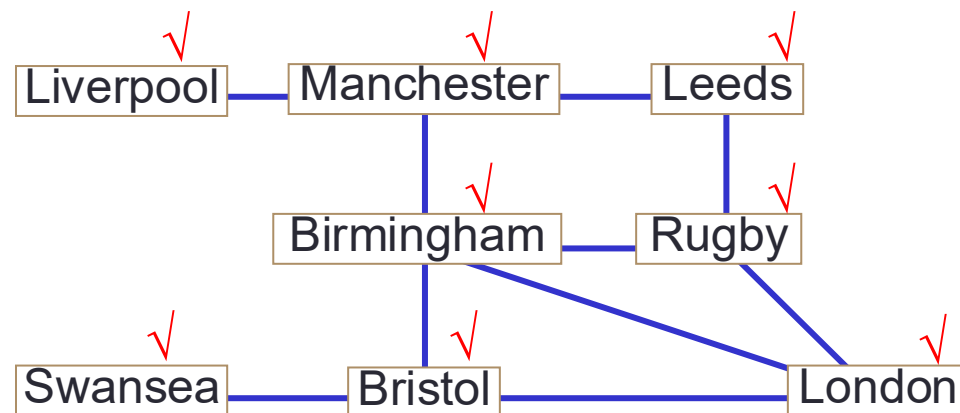
With labelled vertices

Graph traversal (2)

- A graph traversal algorithm could reach the same vertex along several different paths.
- So it must “mark” each vertex the first time it is reached, to avoid visiting the same vertex repeatedly. (Otherwise traversal of a cyclic graph would never terminate.)
- Alternative ways to mark vertices:
 - Store a boolean flag in each vertex, initially false. To mark a vertex, set this flag to true.
 - Maintain a separate set of vertices, initially empty. To mark a vertex, add it to this set.

Depth-first traversal (1)

- **Depth-first traversal** will traverse one possible path as far as possible, before traversing any alternative paths.
- Illustration (starting from London):



Possible order of visits:

London
Bristol
Swansea
Birmingham
Manchester
Liverpool
Leeds
Rugby

Depth-first traversal (2)

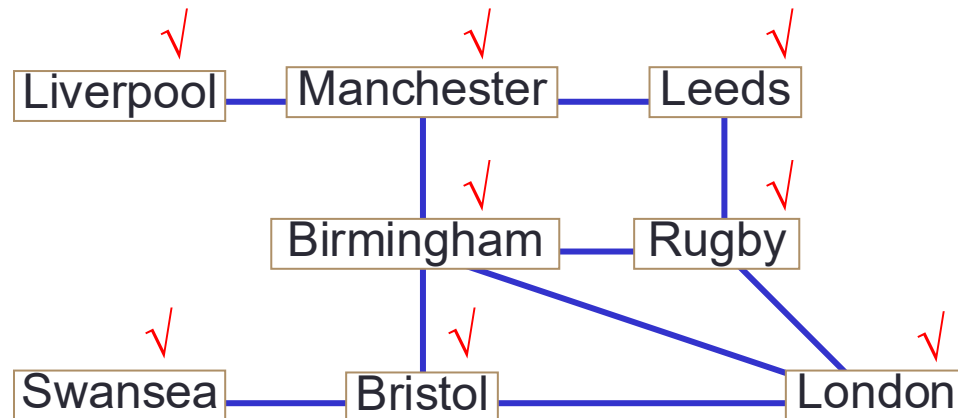
- **Depth-first graph traversal algorithm:**

To traverse graph g in depth-first order, starting at vertex $start$:

1. Make *vertex-stack* contain only vertex $start$, and mark $start$ as reached.
2. While *vertex-stack* is not empty, repeat:
 - 2.1. Remove the top element of *vertex-stack* into v .
 - 2.2. Visit vertex v .
 - 2.3. For each unreached successor w of vertex v , repeat:
 - 2.3.1. Add vertex w to *vertex-stack*, and mark w as reached.
3. Terminate.

Breadth-first traversal (1)

- **Breadth-first traversal** will visit all successors of a vertex, before visiting any of *their* successors. It will traverse shorter paths before longer paths.
- Illustration (starting from London):



Possible order of visits:

London
Bristol
Birmingham
Rugby
Swansea
Manchester
Leeds
Liverpool

Breadth-first traversal (2)

- **Breadth-first graph traversal algorithm:**

To traverse graph g in breadth-first order, starting at vertex $start$:

1. Make *vertex-queue* contain only vertex $start$, and mark $start$ as reached.
2. While *vertex-queue* is not empty, repeat:
 - 2.1. Remove the front element of *vertex-queue* into v .
 - 2.2. Visit vertex v .
 - 2.3. For each unreached successor w of vertex v , repeat:
 - 2.3.1. Add vertex w to *vertex-queue*, and mark w as reached.
3. Terminate.

Graph search (1)

- A **graph search** problem entails finding a path between two given vertices. E.g.:
 - Is there a road route from Glasgow to Dublin?
 - What is the length of the shortest route from Glasgow to London?
- Graph search algorithms are similar to graph traversal algorithms, except that they terminate as soon as a solution is found.

Graph search (2)

- **Depth-first search** will explore one possible path as far as possible, before exploring any alternative paths.
- **Breadth-first search** will explore shorter paths before longer paths.

Shortest path (1)

- Problem: Given a road network, find the shortest path between place *start* and place *dest*.
- Idea:
 - Use breadth-first search.
 - For each place p , let $pred[p]$ be the predecessor of p on the shortest path so far found from *start* to p , and let $dist[p]$ be the distance along that path.
 - Initialize each $dist[p]$ to ∞ (infinity).
 - Thereafter reduce $dist[p]$ (and update $pred[p]$) whenever a shorter path to p is found.

Shortest path (2)

- **Shortest-path algorithm:**

To find the shortest path through a road network from place *start* to place *dest*:

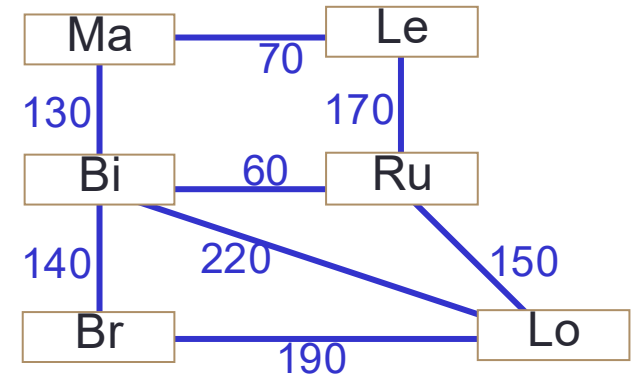
1. Make *places* contain all places in the road network.
2. Set *dist[start]* to 0 and set *pred[start]* to *none*.
3. For each place *p* other than *start*:
 - 3.1. Set *dist[p]* to ∞ and set *pred[p]* to *none*.
4. ...

Shortest path (3)

- Shortest-path algorithm (*continued*):
 4. While *places* is not empty, repeat:
 - 4.1. Remove from *places* the place p with least $dist[p]$.
 - 4.2. If $p = dest$:
 - 4.2.1. Terminate with the path
« $start, \dots, pred[pred[dest]], pred[dest], dest$ ».
 - 4.3. For each road connecting p and another place q ,
such that q is in *places*, repeat:
 - 4.3.1. Let d be $dist[p] + (\text{distance from } p \text{ to } q)$.
 - 4.3.2. If $d < dist[q]$, set $dist[q]$ to d and set $pred[q]$ to p .
 5. Terminate with no path.
- At this point, we have already found the shortest path to *dest*.

Shortest path (4)

- Illustration – shortest path from Lo(ndon) to Le(eds):



<i>dist[Bi]</i> <i>pred[Bi]</i>	Bi	Br	Le	Lo	Ma	Ru	<i>places</i>
	none,∞	none,∞	none,∞	none,0	none,∞	none,∞	{Bi,Br,Le, <u>Lo</u> ,Ma,Ru}
	Lo,220	Lo,190	none,∞	none,0	none,∞	Lo,150	{Bi,Br,Le,Ma, <u>Ru</u> }
	Ru,210	Lo,190	Ru,320	none,0	none,∞	Lo,150	{Bi, <u>Br</u> ,Le,Ma}
	Ru,210	Lo,190	Ru,320	none,0	none,∞	Lo,150	{ <u>Bi</u> ,Le,Ma}
	Ru,210	Lo,190	Ru,320	none,0	Bi,340	Lo,150	{ <u>Le</u> ,Ma}

Shortest path is «Lo, Ru, Le».

Bin-packing (1)

- Suppose we have a number of objects that are to be placed in bins which have fixed capacity
- Objects have different sizes, and we want to use as few bins as possible
...but there is no limit to the number of bins available
- What is the most efficient way to pack the bins? We want to minimise wasted space (and consequently minimise the number of bins required).
- There are various simple algorithms to use
- No one method is best for all data

Bin packing (2)

- Algorithms include:
 - **Best fit:** place next item into the bin with the maximum load it fits into (or open a new bin)
 - **Worst fit:** place next item into the bin with the minimum load it fits into (or open a new bin)
 - **First fit:** place next item into the first bin it will fit into (or open a new bin)
- (there are many more)
- Lower bound on number of bins used is $\lceil (\text{total load}) / \text{capacity} \rceil$
ceiling (a) = smallest integer $\geq a$
- Measure of how well an algorithm packs the bins:
average waste =
$$(\text{total unused space in used bins}) / (\text{number of bins used})$$
- Examples, see over

Example 1

loads = {9,5,6,1,7,2,4,1};

Lower bound for number of bins is 4.00

- Using first fit bins contain: [10, 8, 10, 7]

Average waste is: 1.25

- Using best fit bins contain: [10, 5, 10, 10]

Average waste is: 1.25

- Using worst fit bins contain: [9, 8, 10, 8]

Average waste is: 1.25

Example 2

loads = {1,2,3,4,5,6,7,8,9,5,1,4};

Lower bound for number of bins is 6.00

- Using first fit bins contain: [10, 10, 7, 7, 8, 9, 4]

Average waste is: 2.14

- Using best fit bins contain: [10, 10, 10, 7, 8, 10]

Average waste is: 0.83

- Using worst fit bins contain: [10, 10, 7, 7, 8, 9, 4]

Average waste is: 2.14

See `generalAlgorithms/binPacking.java` (or write your own!)